

Anforderungsliste für Aufgaben 1–3 (Cloud Computing und Big Data)

Hallo Team, im Folgenden findet ihr eine detaillierte Liste der **konkreten Funktionen** für die Aufgaben 1 bis 3 sowie **Umsetzungsvorschläge** mit Terraform und Packer. Die Liste enthält Best Practices und geeignete Tools (z.B. Git zur Versionierung, YAML-Manifeste für Kubernetes), um euch bei der Umsetzung zu unterstützen. Nutzt sie als Leitfaden, um sicherzustellen, dass alle Anforderungen erfüllt sind und euer Vorgehen klar dokumentiert ist.

Aufgabe 1: Immutable Infrastructure

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Technologie auswählen und begründen Wählt ein IaC-Werkzeug (Terraform) und Image-Build-Tool (Packer) und begründet eure Wahl. ¹ (Siehe Aufgabenstellung) „Technologie für Infrastrukturautomatisierung auswählen und Wahl begründen“	– Entscheidet euch für Terraform (Infrastructure as Code) und Packer (Image-Erzeugung) als Kerntechnologien. Begründet in der Dokumentation kurz warum: <i>Terraform</i> erlaubt deklarative Cloud-Provisionierung und konsistente Infrastrukturverwaltung, <i>Packer</i> erstellt reproduzierbare Maschinen-Images. Diese Kombination unterstützt das Immutable-Prinzip hervorragend (Infrastruktur und Images als Code, versionierbar und automatisierbar).
Immutable Komponente entwerfen Entwerft eine einfache Server-Komponente, die unveränderlich ist (z.B. Webserver). ² (Siehe Aufgabenstellung) „einfache Immutable-Komponente entwerfen und Konfiguration + Unveränderlichkeit beschreiben“	– Design eines Basis-Images: Wählt einen einfachen Dienst (z.B. einen Nginx-Webserver oder eine Node.js-App) als <i>Immutable Component</i> . Erstellt mit Packer ein goldenes Image (VM-Image oder Container-Image), das bereits alle nötigen Pakete und die Anwendung enthält. – Konfiguration im Image: Nutzt Packer-Provisioner (Shell-Skripte, Ansible Playbooks o.Ä.), um das Image vorzukonfigurieren (z.B. Installation des Webservers, Bereitstellen der Web-App-Dateien). Dadurch ist die Konfiguration Teil des Images und muss nach dem Start nicht mehr geändert werden. – Unveränderlichkeit sicherstellen: Legt fest, dass keine manuelle Änderung am laufenden System erfolgt. Alle Updates passieren durch Neubau des Images und Neu-Bereitstellung der Instanz, nicht durch Änderung vor Ort. (Tipp: Verwendet eindeutige Versions-IDs oder Tags für Images, sodass Updates als neue Images erkennbar sind und bestehende Instanzen ersetzt werden.)

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Implementierung der Bereitstellung <i>Infrastruktur mittels gewählter Technologie aufsetzen und „Immutable Update“ demonstrieren.</i> ³ (Siehe Aufgabenstellung) „Implementierung mit gewählter Technologie inkl. ‘Immutable Update’ Demonstration“ ³	– Bereitstellung mit Terraform: Schreibt Terraform-Konfigurationen, um eure Infrastruktur bereitzustellen. Beispielsweise könnt ihr auf OpenStack eine VM aus dem Packer-Image starten oder – falls keine Cloud-Umgebung genutzt wird – einen Docker-Container aus dem erstellten Image laufen lassen. Nutzt Variablen, um das Image/den Image-Tag anzugeben, damit ihr leicht zwischen Versionen wechseln könnt. – Immutable Update durchführen: Zeigt die Unveränderlichkeit praktisch, indem ihr z.B. ein Update der Anwendung als neues Image erstellt (Packer build mit neuer Version) und dann via Terraform Apply die alte Instanz/Container durch eine neue Instanz aus dem neuen Image ersetzt. Der alte Stand wird vernichtet und durch den neuen ersetzt – so demonstriert ihr das Prinzip „<i>Pets vs. Cattle</i>“ (Server werden erneuert, nicht manuell gepflegt). – Nutzt Terraform-Best-Practices: behaltet euren Terraform-State konsistent (ggf. in Remote-Backend für Teamzugriff) und verwendet Terraform Outputs, um wichtige Informationen (wie IP-Adresse oder URL des Servers) nach der Provisionierung anzuzeigen ⁴.
Dokumentation & Screencast <i>Schritte, Entwurf und Umsetzung dokumentieren; kurzen Screencast bereitstellen.</i> ⁵ (Siehe Aufgabenstellung) „kurze Dokumentation inkl. Screencast“ ⁵	– Schriftliche Dokumentation: Führt alle Schritte in einer README.md aus. Erklärt zunächst eure Technologieauswahl (warum Terraform/Packer), beschreibt dann den Entwurf der Immutable Infrastructure (Architektur, welche Komponente ihr unveränderlich gestaltet), und dokumentiert die Implementierungsschritte (z.B. Packer-Template erläutern, Terraform-Skripte erklären) ⁶. – Screencast: Nehmt ein kurzes Video auf (einige Minuten, kompakt komprimiert), das eure Umsetzung zeigt. Im Screencast könnt ihr z.B. den Terraform-Prozess beim Bereitstellen des Images zeigen und demonstrieren, wie ein Immutable Update abläuft (altes System heruntergefahren, neues aus dem neuen Image hochgefahren, z.B. indem ihr im Browser die alte vs. neue Version der Anwendung aufruft). Dieser Screencast untermauert eure Dokumentation visuell.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Bonus / Erweiterungen <i>Optional: Unkonventionelle Technologien oder Stateful-Komponenten einsetzen.</i> (Siehe Aufgabenstellung) „Bonus für unkonventionelle Technologien, innovative Entwürfe, Stateful Applications...“ ⁷	– Alternative Tools (optional): Wenn ihr über Terraform/ Packer hinaus experimentieren wollt, könnt ihr auch zusätzliche oder andere Tools einbringen (z.B. Ansible neben Packer für Provisionierung im Image, oder Cloud-Init für VM-Initialisierung). Unkonventionelle Ansätze – etwa Infrastructure-as-Code mit weniger verbreiteten Tools – können Bonuspunkte bringen, sollten aber in der Dokumentation gut erklärt werden. – Stateful Application (optional): Eine besondere Herausforderung im Immutable-Konzept ist der Umgang mit zustandsbehafteten Diensten (Datenbanken, etc.). Wenn ihr z.B. eine kleine Datenbank einbaut, zeigt, wie deren Daten unabhängig vom kurzlebigen Server gespeichert werden (z.B. auf ein externes Volume in OpenStack oder einen Cloud-Datenbankdienst auslagern). Die Fähigkeit, <i>trotz</i> Immutable Infrastructure persistenten State zu managen, würde eure Lösung innovativ machen.

Aufgabe 2: Configuration Management and Deployment

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Infrastrukturdefinition erweitern – App-Installation <i>Ergänzt die vorhandene Immutable Infrastructure um die Installation und Konfiguration einer Beispielanwendung via IaC.</i> (Siehe Aufgabenstellung) „Infrastrukturdefinition um Installation einer Beispielanwendung erweitern“ ⁸	– Packer-Template erweitern: Integriert die Anwendungsinstallation direkt in euer Image. Zum Beispiel könnt ihr im Packer-Build Schritt einen Provisioner nutzen, der den Anwendungscode ins Image kopiert und notwendige Installationen durchführt (etwa <code>npm install</code> für eine Node.js-App oder Installation benötigter Python-Pakete). So stellt ihr sicher, dass die Anwendung beim Start der Instanz bereits vorhanden und konfiguriert ist. <i>Beispiel:</i> Kopiert eure Web-App (z.B. eine einfache Node.js-„Hello World“ oder eine statische HTML-Seite) nach <code>/opt/app</code> und richtet den Webserver ein, diese auszuliefern ⁹. – Alternative (Konfigurationsmanagement): Falls die App nicht ins Image gepackt werden soll, könnt ihr Terraform mit Provisionern (z.B. remote-exec Skripte) oder Tools wie Ansible nutzen, um direkt nach dem Start der VM die Anwendung zu installieren und zu starten. Wichtig ist, dass dieser Prozess vollständig automatisiert und Teil eurer IaC-Beschreibung ist (kein manuelles Nachkonfigurieren). – Start der Anwendung: Stellt sicher, dass die Anwendung nach Deployment läuft. Das kann z.B. bedeuten, einen Service/Daemon einzurichten (bei VM) oder – falls ihr mit Containern arbeitet – dass Terraform den Container der App startet (Terraform Docker-Provider nutzen, um Container bereitzustellen ¹⁰ ¹¹).

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Versionierung der Anwendung demonstrieren <i>Zeigt, wie verschiedene Versionen der Anwendung bereitgestellt und verwaltet werden können.</i> <i>(Siehe Aufgabenstellung) „verschiedene Versionen der Anwendung bereitstellen und verwalten (z.B. Branches im Code-Repo)“</i> ¹²	– Git für Code-Versionierung: Verwendet Git-Banches oder Tags, um verschiedene Versionen eurer Anwendung zu verwalten (z.B. einen Branch <code>v1</code> und <code>v2</code> für zwei Releases der Web-App). In der Dokumentation könnt ihr beschreiben, welche Änderungen die Versionen unterscheiden (z.B. anderer Text auf der Webseite, neue Funktion in der App). – Image-Versionierung: Nutzt Packer-Variablen, um die App-Version ins Image zu übernehmen ¹³ ⁹. Zum Beispiel kann eure Packer-Datei einen Parameter <code>app_version</code> nutzen, um automatisch die richtigen Dateien (für v1 oder v2) ins Image zu kopieren. Bei jedem Build versteht ihr das Image mit einem passenden Tag (z.B. <i>immutable-app:v1</i> vs. <i>v2</i>). – Wechsel der Version in der Infrastruktur: Zeigt praktisch, wie ein Versionswechsel vollzogen wird: Ihr könnt z.B. zunächst Version 1 deployen, anschließend in Terraform eine Variable (oder das Image-Tag) auf Version 2 ändern und <code>terraform apply</code> erneut ausführen. Terraform wird dann die neue Version der Anwendung ausrollen, z.B. indem es den Container mit dem neuen Image startet oder die VM durch eine Instanz mit dem neuen Image ersetzt. Stellt im Screencast dar, dass die neue Version läuft, und erwähnt, wie der Wechsel mittels Versionskontrolle nachvollziehbar bleibt.

Infrastruktur-Versionierung &

Rollback
Versioniert eure Infrastrukturdefinition (Terraform Code) und zeigt, wie man auf ältere Versionen zurückwechselt.

<sup>➡ (Siehe Aufgabenstellung)
 „Infrastrukturdefinition versionieren (z.B. Git) und bestimmte Version installieren oder auf ältere Version zurückrollen; alte Versionen entfernen“ ¹⁴ </sup>

– **Infrastructure as Code in Git:** Haltet eure Terraform-Konfiguration selbst in einem Git-Repository versioniert. Jeder bedeutende Zustand der Infrastruktur (z.B. „Infra mit App v1“, „Infra mit App v2“) kann durch einen Git-Tag oder unterschiedliche Branches repräsentiert werden. Dokumentiert, wie der **Git-Verlauf** genutzt wird, um Änderungen nachzuvollziehen.
– **Rollout einer bestimmten Version:** Demonstriert, wie ihr z.B. durch Auschecken eines älteren Git-Commits eurer Terraform-Codebasis und Ausführung von `terraform apply` exakt die frühere Infrastruktur wiederherstellen könnt. Terraform wird dann ggf. die neuere Version der Ressourcen löschen und die ältere Version erneut bereitstellen.
– **Rollback und Aufräumen:** Zeigt auch, wie **Ressourcen alter Versionen entfernt** werden, um keine unnötigen Kosten/Ressourcen zu verursachen ¹⁵. In einem Cloud-Kontext hieße das z.B., nicht mehr benötigte VMs abzuschalten oder zu zerstören. In eurem Beispiel (falls lokal mit Docker) könnte das bedeuten, den alten Container zu stoppen/löschen, sobald die neue Version läuft. Automatisiert das Aufräumen nach Möglichkeit in euren Terraform-Skripten (Terraform zerstört von sich aus gelöschte Ressourcendefinitionen bei `apply`).

Dokumentation der

Erweiterungen
Detaillierte Doku aller neuen Schritte (App-Bereitstellung, Versionierungs- und Rollback-Vorgehen).

<sup>➡ (Siehe Aufgabenstellung)
 „erweiterte Infrastrukturdefinition und Schritte zur App-Bereitstellung/Versionierung und Rollback dokumentieren“ ¹⁶ </sup>

– **Ergänzt die README:** Beschreibt zunächst die **Integration der Anwendung in die Infrastruktur** – z.B. „Wir haben das Packer-Image um die App-Installation erweitert...“. Zeigt Code-Auszüge eurer Terraform-/Packer-Konfiguration, die für die App relevant sind (etwa das Packer-File für App v1/v2, Ausschnitte aus Terraform, die Container starten oder Provisioner ausführen). Erklärt dabei, wie durch diese Codeänderungen die App automatisch installiert wird.
– **Versionierung & Rollback erklären:** Dokumentiert euer Git-Branching/Tagging-Modell für die App und Infrastruktur. Zum Beispiel: „Version 2 der Anwendung liegt auf Branch 'v2'; um diese zu deployen, haben wir das Terraform-Variable X auf 'v2' gesetzt und neu deployed.“ Beschreibt Schritt für Schritt, wie ein Rollback auf Version 1 ablaufen würde (Git alten Stand auschecken, `terraform apply` → System wechselt zurück auf v1).
– **Screenshots & Outputs:** Nutzt ggf. Screenshots oder Kommandoausgabe-Auszüge, um zu zeigen, wie Terraform meldet, was geändert wird (z.B. dass ein neuer Container erstellt und der alte gelöscht wird). Diese visuelle Unterstützung erhöht die Nachvollziehbarkeit.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Präsentation & Screencast Abschließende Zusammenfassung mit Live-Demo der Versionierung und Deployment-Prozesse.
<sup>➡ (Siehe Aufgabenstellung) „Dokumentation inkl. Screencast als Präsentation vorführen“ 17 </sup>	– Screencast für Task 2: Nehmt ein Video auf, das die Kernfunktionen dieser Aufgabe zeigt: z.B. den Ablauf eines Deployments der App auf die bestehende Infrastruktur, dann ein Update auf eine neue Version und ggf. den Rollback. Zeigt in der Aufnahme z.B. Terminal-Sessions: erst <code>terraform apply</code> für Version 1 (App kommt hoch), dann Wechsel auf Version 2 (neues Image bauen, Terraform apply zeigt das Ersetzen der Ressourcen) und anschließend, falls möglich, einen Rollback (wieder <code>git checkout</code> der älteren Version und <code>terraform apply</code>, um zur Version 1 zurückzukehren).
– Live-Demo in Präsentation: Bereitet euch darauf vor, diese Schritte auch live oder zumindest im Gespräch zu erklären. Fokussiert auf das Zusammenspiel <i>Infrastruktur</i> <-> <i>Anwendung</i>: Es soll klar werden, dass eure Infrastruktur-Code und App-Code versioniert sind und zusammenarbeiten. Die Zuschauer/Prüfer sollten nachvollziehen können, wie ihr durch reines Umschalten von Versionsständen in Code die tatsächliche Umgebung verändert. (Tipp: Nutzt in der Präsentation Folien, die euer Vorgehen zusammenfassen, und das Video als Beleg der funktionierenden Implementierung.)
Bonus / Unkonventionelles Optional: Einsatz ungewöhnlicher Tools oder besonders effizienter Abläufe.
<sup>➡ (Siehe Aufgabenstellung) „Bonus für unkonventionelle Technologien, innovative Entwürfe...“ 18 </sup>	– Alternative Deployment-Strategien: Ihr könnt Bonuspunkte sammeln, wenn ihr z.B. einen CI/CD-Ansatz zeigt: Etwa automatisches Bauen und Verteilen der neuen Version per Pipeline (z.B. GitHub Actions, die Packer und Terraform anstoßen, sobald ein neuer Tag gepusht wird). Dies geht über die Vorlesungsinhalte hinaus und demonstriert Eigeninitiative.
– Andere Tools einbinden: Falls in der Veranstaltung z.B. nur Ansible gezeigt wurde, ist der Einsatz von Terraform/ Packer an sich schon eine eigenständige Entscheidung. Weitere unübliche Kombinationen (z.B. Terraform + Chef oder Puppet für Konfigurationsmanagement, oder Verwendung von Docker-Containern als Infrastructure Component wie in Aufgabe 1) könnt ihr hervorheben. Wichtig ist, dass ihr begründet, warum das sinnvoll ist und welchen Mehrwert es bringt.

Aufgabe 3: Microservice-Architektur

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Kubernetes-Cluster in OpenStack einrichten Wählt eine geeignete Technologie, um eine Multi-Node-Kubernetes-Umgebung auf OpenStack bereitzustellen. [➡] (Siehe Aufgabenstellung) „Technologie zur Installation der Kubernetes-Umgebung in OpenStack wählen (z.B. kubectl, Rancher, kOps)“ ¹⁹	– Technologieauswahl für K8s: Entscheidet euch für einen Weg, Kubernetes auf OpenStack aufzusetzen. Mögliche Ansätze: ① kubeadm: Nutzt Terraform, um die benötigten VMs (Master und Worker Nodes) in OpenStack zu erstellen. Anschließend könnt ihr mittels Cloud-Init-Scripts oder manueller Schritte (automatisiert per SSH/remote-exec oder Ansible) auf der Master-VM <code>kubeadm init</code> ausführen und die Worker mit <code>kubeadm join</code> verbinden. Terraform kann hierbei Skripte triggern, um diesen Prozess zu orchestrieren. ② Rancher: Alternativ könnt ihr eine Rancher-Server-Instanz starten (per Terraform als VM oder Docker-Container) und darüber ein Kubernetes-Cluster in OpenStack verwalten. Rancher bietet eine Web-GUI zur einfachen Einrichtung von Clustern – dies wäre eine etwas <i>unkonventionelle</i> Wahl und könnte Bonus einbringen, muss aber gut dokumentiert werden. ③ kOps oder Terraform-Provider: <i>kOps</i> unterstützt OpenStack und könnte den Cluster-Aufbau automatisieren, oder ihr nutzt den Terraform OpenStack-Provider plus Terraform Kubernetes-Provider gemeinsam: Zuerst erstellt ihr die OpenStack-Ressourcen (Netzwerk, VMs) mit Terraform, dann könnt ihr Terraform dazu nutzen, Kubernetes-Ressourcen zu definieren (z.B. via Helm-Charts) – dieser Ansatz ist jedoch komplex. – Begründung & Best Practices: Legt dar, warum eure gewählte Methode sinnvoll ist (z.B. „Wir verwenden <i>kubeadm</i>, da es der Standard für selbstverwaltete Cluster ist und wir die volle Kontrolle lernen wollen.“). Achtet auf Best Practices bei der Cluster-Installation: z.B. sichert den Zugriff auf den Cluster (Kubeconfig nur für Admins), verwendet passende VM-Größen in OpenStack für Master/Worker, und verteilt die Worker auf mehrere Availability Zones (falls verfügbar), um Hochverfügbarkeit zu fördern.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Anwendung containerisieren <i>Verpackt eure Anwendung in einem Docker-Container und stellt sicher, dass sie versionierbar ist.</i> <i>
<sup>➡ (Siehe Aufgabenstellung) „Anwendung containerisiert und versionierbar auf dem Cluster deployen“ 20 </sup></i>	– Docker-Image bauen: Erstellt ein Docker-Image eurer Anwendung. Falls ihr bereits Packer in Aufgabe 2 dafür genutzt habt (z.B. um ein Docker-Image mit nginx+Web-App zu bauen), könnt ihr dieses Image oder ein ähnliches wiederverwenden. Andernfalls schreibt ein Dockerfile für eure App (z.B. für eine Node.js-App eine Base-Image <code>node:18-alpine</code>, Code reinkopieren, <code>npm install</code> ausführen, App starten). – Image-Versionierung: Taggt das Docker-Image mit einer Versionsnummer (z.B. <code>myapp:v1</code>, <code>v2</code>). Nutzt ein Container-Registry (Docker Hub oder eine private Registry), um die Images zu speichern und für Kubernetes verfügbar zu machen. (Tipp: Für lokale Tests könnt ihr Images auch auf den Nodes vorab laden, aber eine Registry ist professioneller.) – Deployment in Kubernetes: Schreibt eine Kubernetes Deployment-Definition (YAML), die eure containerisierte Anwendung im Cluster ausrollt. Referenziert darin das Docker-Image mit Tag. Setzt zunächst <code>replica=1</code>. Das Deployment-Objekt erleichtert später Updates: um Version 2 auszurollen, ändert man einfach den Image-Tag und Kubernetes macht ein Rolling Update. – Versionierbarkeit demonstrieren: Um zu zeigen, dass eure Anwendung versionierbar deployed werden kann, könnt ihr analog zu Aufgabe 2 zunächst Version 1 ins Cluster deployen. Anschließend erzeugt ihr Version 2 (neues Docker-Image pushen) und aktualisiert das Deployment (<code>kubectl apply</code> mit geänderter YAML oder <code>kubectl set image</code>). Kubernetes wird dann die Pods der neuen Version starten und die alten beenden (<i>Rolling Update</i>). Dokumentiert diesen Vorgang und stellt im Screencast einen Vorher/Nachher-Vergleich an (z.B. Response der App zeigt Version).

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Skalierbarkeit der Anwendung <i>Sorgt dafür, dass die Anwendung skalierbar ist – horizontal in Kubernetes.</i>
<sup>➡ (Siehe Aufgabenstellung) „Anwendung soll skalierbar sein“ 20 </sup>	– Replicas hochskalieren: Konfiguriert euer Deployment so, dass es mehrere Replikate der Anwendung unterstützen kann. Testet z.B., die <code>replicas</code> in der Deployment-YAML auf 3 zu setzen und wendet dies an. Kubernetes sollte dann drei Pods der Anwendung laufen haben. Verifiziert, dass der Traffic (z.B. via Service) auf alle verteilt wird.
– Load Balancing & Service: Erstellt einen Service vom Typ ClusterIP für eure Anwendung, der die Pods anspricht. Kubernetes verteilt Anfragen an den Service automatisch auf die Pods (Round-Robin). In eurer Dokumentation solltet ihr den Zusammenhang zwischen <i>Service</i> und <i>Pods</i> erläutern, da dies zentral für Skalierbarkeit ist (Clients sprechen den Service an, nicht einzelne Pod-IP).
– Optional Auto-Scaling: Wenn ihr mehr zeigen wollt, richtet einen Horizontal Pod Autoscaler (HPA) ein, der je nach Last die Anzahl der Pods automatisch anpasst. Dazu benötigt ihr in der Regel Metrics-Server im Cluster. Ihr könnt z.B. einen HPA definieren, der bei hoher CPU-Last von 3 auf bis zu 5 Pods erhöht. Diese Feinheit ist optional, demonstriert aber fortgeschrittenes Verständnis.
– Skalierung der Cluster-Infrastruktur: Erwähnt, wie man den Cluster selbst skalieren könnte (z.B. weitere Worker-Knoten per Terraform hinzufügen, falls Last sehr steigt). Dieser Schritt muss nicht unbedingt praktisch durchgeführt werden, aber das Verständnis, dass sowohl Infrastruktur skalieren (mehr VMs) als auch Applikation skalieren (mehr Pods) möglich ist, sollte durchscheinen.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Externe Erreichbarkeit (Ingress) <i>Stellt sicher, dass die Anwendung von außerhalb des Clusters erreichbar ist, z.B. über einen Ingress.</i> <i>
<sup>➡ (Siehe Aufgabenstellung) „über Ingress von außen erreichbar“ 20 </sup></i>	– Ingress-Controller installieren: Wählt einen Ingress-Controller (häufig NGINX Ingress Controller). Diesen könnt ihr via YAML/Helm im Cluster installieren. Falls euer OpenStack keine LoadBalancer-API bietet, nutzt HostPort/NodePort oder MetalLB (LoadBalancer in Cluster) in Kombination mit Floating IPs. Terraform kann euch z.B. bereits beim VM-Launch eine Floating IP für den Master oder einen LoadBalancer-VM reservieren. – Ingress Resource anlegen: Schreibt eine Ingress-YAML, die einen Hostnamen oder Pfad definiert und auf euren Service zeigt. Beispielsweise könnt ihr <code>yourapp.example.com</code> auf den Service der Anwendung routen lassen (falls ihr keinen echten Domainnamen habt, könnt ihr in der Demo auch mit der nackten IP + Port 80 arbeiten oder <code>/etc/hosts</code> Eintrag). – Test der Erreichbarkeit: Verifiziert von außerhalb (z.B. von eurem Rechner oder einer VM außerhalb des Cluster-Netzes), dass die Anwendung erreichbar ist. Im Screencast könnt ihr zeigen, wie ihr über den Browser oder <code>curl</code> die bereitgestellte URL/Adresse aufruft und die App antwortet. Das beweist, dass euer Ingress-Setup funktioniert. – Sicherheit beachten: Erwähnt ggf., dass man in Produktionsumgebungen TLS für Ingress einrichten würde (z.B. via Let's Encrypt), auch wenn ihr es im Test evtl. nicht umsetzt. Dies zeigt Bewusstsein für sichere externe Zugänge.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Optional: Monitoring mit Prometheus <i>Überwacht die Anwendung und/oder den Cluster mit Prometheus (Bonus).</i> (Siehe Aufgabenstellung) „Performance per Prometheus überwacht (Bonus)“ ²¹	– Prometheus Deployment: Setzt (falls Ressourcen und Zeit vorhanden) einen Prometheus-Server im Cluster auf. Am einfachsten geht dies mit dem Prometheus Operator oder einem fertigen Helm Chart, das Prometheus + Grafana deployt. Terraform kann hier unterstützend wirken, indem es Helm-Charts via Kubernetes-Provider installiert, oder ihr verwendet wieder manuell <code>kubect1</code> /Helm. – Metriken sammeln: Konfiguriert Prometheus so, dass es entweder Kubernetes-Metriken (Cluster-Stats) einsammelt und/oder Metriken eurer Anwendung (falls diese Metriken ausgeben kann, z.B. über einen <code>/metrics</code> Endpoint oder via Exporter). Für den Anfang reicht es, wenn Prometheus grundlegende Node-/Pod-Metriken sammelt. – Visualisierung: Falls ihr Grafana deployt (oft im Prometheus Helm Chart enthalten), richtet ein einfaches Dashboard ein, das z.B. CPU-/RAM-Auslastung der App-Pods zeigt oder die Anzahl Requests (sofern gemessen). In der Dokumentation könnt ihr einen Screenshot des Dashboards einfügen. – Mehrwert hervorheben: Erklärt, wie Monitoring hilft (Performance beobachten, Engpässe erkennen, Auto-Scaling optimieren). Da dies optional ist, bringt ein funktionierendes Monitoring Bonuspunkte – stellt also sicher, dass es soweit läuft, dass ihr im Präsentationsvideo kurz darauf eingehen könnt (z.B. “Hier sieht man in Grafana die Auslastung unseres Pods während des Tests...”).

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Dokumentation: Installation, Container, K8s-Konfiguration Alle Schritte von Cluster-Setup bis Deployment und Scaling schriftlich festhalten. (Siehe Aufgabenstellung) „Dokumentation der Installationsschritte, Containerisierung, Kubernetes-Konfigurationen (Deployment, Service, Ingress), Versionierungs- und Skalierungsstrategien...“ ²²	– Cluster-Einrichtung: Dokumentiert, wie ihr den Kubernetes-Cluster eingerichtet habt. Beschreibt die gewählte Methode („Wir haben mithilfe von Terraform X VMs erstellt und dann mit kubectl initialisiert...“ oder „Wir haben Rancher verwendet...“). Listet wichtige Terraform-Konfigurationsschritte (z.B. Terraform-Modul für OpenStack-Netzwerk, Sicherheitsgruppen für K8s-Ports, etc.). Erwähnt Probleme und Lösungen (z.B. benötigte OpenStack-Neutron-Einstellungen für Load Balancer, falls relevant). – Containerisierung: Erklärt, wie das Docker-Image der Anwendung gebaut wurde. Gebt den Dockerfile-Inhalt oder Packer-Schritte an, die das Image erzeugen. Zeigt, wie Versions-Tags verwendet wurden. Diese Beschreibung sollte verdeutlichen, dass die Anwendung unabhängig von der Infrastruktur als Container vorliegt und somit portabel und versionierbar ist. – Kubernetes-Manifeste: Legt eure YAML-Dateien für Deployment, Service und Ingress in die Doku (oder verlinkt darauf) und erläutert sie. Zum Beispiel: „Das Deployment-Manifest definiert 3 Replikate unserer App und nutzt das Image <code>myapp:v2</code>. Der zugehörige Service exponiert Port 80 im Cluster... Unser Ingress leitet Traffic von <code>/</code> an diesen Service weiter...“. Durch diese Erklärungen zeigt ihr Verständnis der K8s-Ressourcen. – Versionierungs- & Skalierungsstrategie: Beschreibt, wie ihr Versionen managed (Docker-Image-Tags, Rolling Updates via Deployment) und wie ihr skaliert (manuelles Ändern der Replica-Zahl, ggf. Autoscaling über HPA). Diese Strategien sollten in der Theorie erläutert und in der Praxis (durch euer Setup) belegbar sein. – Optional Monitoring: Falls umgesetzt, dokumentiert die Einrichtung von Prometheus/Grafana: Welche Komponenten installiert, wie konfiguriert, und welche Metriken überwacht werden. Hebt hervor, dass dies zur Performance-Überwachung dient und z.B. in Kombination mit Autoscaling nützlich sein kann.

Funktion / Anforderung	Umsetzungsvorschlag (Terraform, Packer & Tools)
Präsentation & Demo <i>Funktionsweise des Kubernetes-Deployments, Versionierungs- und Skalierungsfeatures live vorführen.</i>
<sup>➡ (Siehe Aufgabenstellung) „Präsentation demonstriert Kubernetes-Umgebung, Anwendungsdeployment, Versionierung, Skalierbarkeit, externe Erreichbarkeit (und ggf. Monitoring)“ 22 </sup>	– Live-Demo/ScreenCast für Task 3: Zeigt in eurer Abschlusspräsentation die laufende Kubernetes-Umgebung und die wichtigsten Ergebnisse:
 ① Deployment & Ingress: Demonstriert, dass eure Anwendung im Browser oder via <code>curl</code> von außen erreichbar ist (Ingress funktioniert).
 ② Skalierung: Skaliert live die Anwendung (z.B. repliziert auf mehrere Pods via <code>kubectl scale</code> oder zeigt bereits laufende 3 Pods) und zeigt, wie die Last verteilt wird. Vielleicht könnt ihr zwei Browserfenster mit den Pod-IPs öffnen, um zu veranschaulichen, dass verschiedene Pods antworten – oder nutzt ein Tool, das die Service-Round-Robin verdeutlicht.
 ③ Versionsierung: Wechselt die Anwendungsversion live: z.B. führt ein <code>kubectl set image</code> Befehl aus oder wendet ein geändertes Deployment-Manifest an, um von Version 1 auf Version 2 zu aktualisieren. Beobachtet, wie Kubernetes alte Pods terminiert und neue startet. Die Außenwelt bekommt keinen nennenswerten Ausfall – ein Vorteil von Kubernetes Rolling Updates.
 ④ (Optional) Monitoring: Falls implementiert, zeigt kurz das Prometheus/Grafana-Dashboard mit ein paar Metriken, während Last generiert wird, um zu beweisen, dass Monitoring läuft.
– Präsentationsstruktur: Erklärt zu jedem gezeigten Schritt, was gerade passiert und warum es wichtig ist. Zielt darauf ab, dem Publikum ein Verständnis dafür zu vermitteln, wie eure Cloud-Infrastruktur (via Terraform), Immutable Images (via Packer), Containerisierung und Kubernetes zusammenwirken, um eine skalierbare, versionierbare Microservice-App bereitzustellen. Abschließend fasst noch einmal die Vorteile eurer gewählten Ansätze zusammen (Automatisierung, Zuverlässigkeit, Skalierbarkeit).

Viel Erfolg bei der Umsetzung! Denkt daran, dass **Sauberkeit in der Ausführung und Klarheit in der Dokumentation** genauso wichtig sind wie die funktionierende Technik. Mit dieser Anforderungsliste könnt ihr Schritt für Schritt prüfen, ob alle Aspekte abgedeckt sind – so ist euer Team bestens aufgestellt, um die Aufgaben 1–3 erfolgreich abzuschließen.

1 2 3 5 6 7 8 12 14 15 16 17 18 19 20 21 22 Modul_Cloud Computing und Big Data_ - Aufgabenstellung - SS2025.pdf

file:///file-RTL4eH3mBGjmPviuCBgejh

4 10 11 main.tf

<https://github.com/xWaqez/Cloud-Computing-Big-Data/blob/61413e3ea1c76c3ecbdf7b9919d7f811fa2bea20/>

Aufgabe2_ConfigManagementDeployment/Terraform/main.tf

9 13 webserver.pkr.hcl

<https://github.com/xWaqez/Cloud-Computing-Big-Data/blob/61413e3ea1c76c3ecbdf7b9919d7f811fa2bea20/>

Aufgabe1_ImmutableInfrastructure/packer/webserver.pkr.hcl